Adaptive Low-Rank Hessian Approximation for Large-Scale Optimization: Theory, Algorithms, and Applications

Arush Rao Lagudu

arushlagudu@gmail.com
Frisco Centennial High School

Abstract

We introduce the Adaptive Low-rank Hessian Approximation (ALHA) algorithm, a novel quasi-Newton method that dynamically adjusts the rank of the Hessian approximation based on local curvature information and optimization progress. Unlike classical limited-memory BFGS (L-BFGS), which requires manual selection of a fixed memory parameter m, ALHA automatically adapts its effective rank throughout optimization using a dual signal: a held-out spectral approximation quality metric and a gradient norm progress rate. This eliminates the need for problem-specific tuning of m.

We establish rigorous convergence guarantees under standard assumptions: for L-smooth and μ-strongly convex functions, ALHA achieves linear convergence at rate $O\big((1-\mu/L)^k\big)$, matching the optimal rate for first-order methods with curvature information. Our analysis introduces a novel held-out spectral approximation quality metric that governs rank adaptation and provides explicit bounds on the approximation error.

Comprehensive numerical experiments on six diverse optimization problems—including quadratic functions, the Rosenbrock function, logistic regression on real MNIST data, neural network training, and sparse logistic regression—demonstrate that ALHA automatically uses low rank on well-conditioned problems (preserving efficiency) and high rank on hard problems (preserving accuracy), all without manual tuning. On logistic regression with real MNIST (n=12,665, d=784), ALHA achieves the fastest wall-clock time (1.40 s vs. 1.94 s for best fixed-rank L-BFGS) while using average rank 8.6 compared to m=20. On neural network training (2-layer MLP, d=39,760 parameters), ALHA reaches 100% training accuracy 3.2–3.6× faster than all fixed-rank L-BFGS variants.

Keywords: Quasi-Newton methods, L-BFGS, adaptive algorithms, low-rank approximation, large-scale optimization, convergence analysis, Hessian approximation

1 Introduction

1.1 Motivation and Problem Setting

Large-scale optimization problems arise ubiquitously in machine learning, scientific computing, and engineering. Consider the unconstrained minimization problem

$$\min_{x\in\mathbb{R}^d} f(x), \tag{1}$$

where $f:\mathbb{R}^d\to\mathbb{R}$ is twice continuously differentiable and d may range from thousands to billions. First-order methods require only $O(d)$ operations per iteration but suffer from slow convergence on ill-conditioned problems, with complexity scaling as $O(\kappa\log(1/\varepsilon))$ where $\kappa = L/\mu$ is the

condition number. Newton's method achieves quadratic local convergence but requires $O(d^3)$ operations per iteration.

Quasi-Newton methods, particularly L-BFGS [Nocedal, 1980, Liu and Nocedal, 1989], occupy a middle ground by constructing low-rank approximations to the Hessian using only gradient information. L-BFGS maintains a fixed memory parameter m (typically $m \in \{3, 5, 10, 20\}$) that determines the rank of the approximation, achieving $O(md)$ per-iteration cost.

However, the choice of m presents a fundamental trade-off:

- **Small** m: Low computational cost but potentially poor Hessian approximation, leading to slow convergence on ill-conditioned problems.
- **Large** m: Better approximation but increased memory and computational overhead.

This trade-off is problem-dependent and requires manual tuning. Moreover, the optimal rank may vary *during* optimization.

1.2 Our Contributions

We propose ALHA, which addresses these limitations through dynamic rank adaptation. Our main contributions are:

1. **Novel Algorithm:** ALHA adapts rank using a dual signal combining a held-out spectral quality metric and a gradient norm progress rate. Rank increases when either signal indicates poor approximation or slow progress; rank decreases only when both signals agree quality is sufficient.
2. **Convergence Theory:** Linear convergence at rate $O\big((1-\mu/L)^k\big)$ for L-smooth, μ-strongly convex functions, with complete proofs.
3. **Approximation Quality Bounds:** Tight bounds relating the held-out spectral quality to the convergence rate.
4. **Comprehensive Experiments:** Validation on six diverse problems including real MNIST data, with key results: fastest wall-clock time on logistic regression, 3.2–3.6× faster neural network training than fixed-rank L-BFGS, and automatic rank adaptation matching problem difficulty.

1.3 Paper Organization

Section 2 reviews related work. Section 3 establishes notation and background. Section 4 presents ALHA. Section 5 contains convergence theory. Section 6 analyzes computational complexity. Section 7 presents experiments. Section 8 discusses limitations and extensions. Section 9 concludes.

2 Related Work

Classical Quasi-Newton Methods. Quasi-Newton methods date to Davidon [1959], Fletcher and Powell [1963], and Broyden [1970]. The BFGS update constructs positive definite Hessian approximations via rank-two updates. Dennis and Moré [1974] and Dennis and Schnabel [1983] established superlinear convergence; Powell [1976] analyzed global convergence with inexact line searches.

Limited-Memory Methods. Nocedal [1980] introduced L-BFGS, storing only the m most recent curvature pairs and computing search directions via the two-loop recursion. Liu and Nocedal [1989] provided extensive comparisons. Berahas et al. [2022] established explicit convergence rates with dependence on m. Rodomanov and Nesterov [2021] analyzed greedy quasi-Newton methods with superlinear rates.

Adaptive and Self-Tuning Methods. Duchi et al. [2011] introduced AdaGrad; Kingma and Ba [2015] proposed Adam. Morales and Nocedal [2002] proposed automatic scaling for L-BFGS. Most relevant is Zhao et al. [2024], whose Adapprox method uses adaptive low-rank approximation for Adam's second-moment estimates. Our work extends this principle to quasi-Newton Hessian approximations with rigorous convergence guarantees.

Positioning. ALHA differs from existing methods in that: (1) unlike L-BFGS with fixed m, it adapts rank without manual tuning; (2) unlike Adapprox, it operates on quasi-Newton Hessian approximations; (3) unlike randomized methods [Pilanci and Wainwright, 2017], it uses deterministic rank adaptation; and (4) we provide complete convergence analysis.

3 Preliminaries

3.1 Notation

Throughout: $\mathbb{R}^d$ denotes d-dimensional Euclidean space; $\|\cdot\|$ denotes the Euclidean norm for vectors and spectral norm for matrices; $\|\cdot\|_F$ denotes the Frobenius norm; $\lambda_{\min}(A)$, $\lambda_{\max}(A)$ denote minimum and maximum eigenvalues of symmetric A; $\kappa(A) = \lambda_{\max}(A)/\lambda_{\min}(A)$ is the condition number; $A \succ 0$ ($A \succeq 0$) means A is positive definite (semidefinite); I_d is the $d \times d$ identity matrix.

3.2 Assumptions

Assumption 1 (Smoothness)**.** *$f : \mathbb{R}^d \to \mathbb{R}$ is twice continuously differentiable with L-Lipschitz gradient: $\|\nabla f(x) - \nabla f(y)\| \le L \|x - y\|$ for all $x, y \in \mathbb{R}^d$.*

Assumption 2 (Strong Convexity)**.** *f is μ-strongly convex with $\mu > 0$: $f(y) \ge f(x) + \langle \nabla f(x), y - x\rangle + \frac{\mu}{2}\|y - x\|^2$ for all $x, y \in \mathbb{R}^d$.*

Under Assumptions 1–2, the condition number is $\kappa = L/\mu \ge 1$ and there exists a unique minimizer x^*.

3.3 The L-BFGS Algorithm

L-BFGS maintains the m most recent curvature pairs $\{(s_k, y_k)\}$ where $s_k = x_{k+1} - x_k$ and $y_k = \nabla f(x_{k+1}) - \nabla f(x_k)$. The search direction $d_k = -H_k \nabla f(x_k)$ is computed via the two-loop recursion in $O(md)$ operations, where H_k implicitly approximates $[\nabla^2 f(x_k)]^{-1}$.

3.4 Wolfe Line Search Conditions

We employ a line search satisfying the strong Wolfe conditions:

$$f(x_k + \alpha_k d_k) \le f(x_k) + c_1 \alpha_k \nabla f(x_k)^\top d_k, \tag{2}$$

$$|\nabla f(x_k + \alpha_k d_k)^\top d_k| \le c_2 |\nabla f(x_k)^\top d_k|, \tag{3}$$

where $0 < c_1 < c_2 < 1$ (typically $c_1 = 10^{-4}$, $c_2 = 0.9$).

4 The ALHA Algorithm

4.1 Motivation and Key Ideas

The central insight is that the optimal rank for Hessian approximation varies during optimization and across problems. We introduce two complementary signals for rank adaptation:

- **Held-out spectral quality** (Section 4.2): measures how well the current approximation satisfies the secant condition on held-out curvature pairs.
- **Gradient norm progress rate** (Section 4.3): measures whether convergence is proceeding fast enough; slow progress signals that higher rank is needed.

Combining both signals makes rank adaptation robust: quality alone can be unreliable on highly ill-conditioned problems due to noise in individual curvature pairs, while progress rate alone can be too aggressive. Together they provide a stable and responsive rank controller.

4.2 Held-Out Spectral Approximation Quality

Definition 4.1 (Held-Out Quality Metric)**.** *Given the current set of stored curvature pairs $\mathcal{S} = \{(s_i, y_i)\}$, the held-out spectral approximation quality is:*

$$\hat{Q}_k = \frac{(H_{\text{train}}\, y_{\text{val}})^\top y_{\text{val}}}{s_{\text{val}}^\top y_{\text{val}}}, \tag{4}$$

where $(s_{\text{val}}, y_{\text{val}}) = (S[0], Y[0])$ is the oldest stored pair (held out for validation), and H_{train} is the L-BFGS inverse Hessian approximation built from the remaining pairs $S[1:], Y[1:]$.

Why held-out? The naive approach of computing $\hat{Q}_k = (H_k y_k)^\top y_k / (s_k^\top y_k)$ using the *current* pair (s_k, y_k)—which was just added to the approximation—inflates quality estimates and produces $\hat{Q}_k \approx 1$ regardless of true approximation quality. This renders the rank adaptation mechanism ineffective, as the algorithm never detects poor quality and rank stays at $m_{\min}$. The held-out metric provides a true out-of-sample quality estimate.

For a perfect approximation $H_k = [\nabla^2 f(x_k)]^{-1}$, we have $\hat{Q}_k = 1$. When $\hat{Q}_k \ll 1$, the approximation fails to satisfy the secant condition on unseen pairs, indicating higher rank is needed.

4.3 Progress Rate Signal

Definition 4.2 (Gradient Norm Progress Rate)**.** *The progress rate at iteration k with window w is:*

$$\rho_k = \frac{\|\nabla f(x_k)\|}{\|\nabla f(x_{k-w})\|}. \tag{5}$$

If $\rho_k > \rho_{\text{slow}}$ (default 0.95), convergence is too slow and rank should increase. If $\rho_k < \rho_{\text{fast}}$ (default 0.7), convergence is fast and rank may decrease.

4.4 Rank Adaptation Mechanism

ALHA adapts rank m_k using a conservative asymmetric rule:

$$m_{k+1} = \begin{cases} \min(m_k + \Delta m^+,\, m_{\max}) & \text{if } \hat{Q}_k < 1 - \varepsilon_{\text{high}} \;\text{ or }\; \rho_k > \rho_{\text{slow}}, \\ \max(m_k - \Delta m^-,\, m_{\min}) & \text{if } \hat{Q}_k > 1 - \varepsilon_{\text{low}} \;\text{ and }\; \rho_k < \rho_{\text{fast}}, \\ m_k & \text{otherwise.} \end{cases} \tag{6}$$

The asymmetry is intentional: rank increases if *either* signal is poor (conservative), but decreases only if *both* agree quality is sufficient (cautious). This prevents premature rank reduction on difficult problems.

Algorithm 1 Adaptive Low-Rank Hessian Approximation (ALHA)

Require: $x_0 \in \mathbb{R}^d$, m_0=5, $m_{\min}$=2, $m_{\max}$=50
Require: ε_{low}=0.1, $\varepsilon_{\text{high}}$=0.3, Δm^+=2, Δm^-=1
Require: c_1=10^{-4}, c_2=0.9, w=3, ρ_{fast}=0.7, ρ_{slow}=0.95

```
S, Y ← ∅;  g_0 ← ∇f(x_0);  γ ← 1;  grad_window ← [‖g_0‖]
for k = 0, 1, 2, ..., K_max do
    if ‖g_k‖ < ε_tol then return x_k
    end if
    d_k ← −TwoLoopRecursion(g_k, S, Y, m_k, γ)
    if g_k^⊤ d_k ≥ 0 then d_k ← −g_k
    end if                                          ▷ fallback to steepest descent
    α_k ← StrongWolfeLineSearch(f, x_k, d_k)
    x_{k+1} ← x_k + α_k d_k;  g_{k+1} ← ∇f(x_{k+1})
    s_k ← x_{k+1} − x_k;  y_k ← g_{k+1} − g_k
    if s_k^⊤ y_k > ε_curv ‖s_k‖ ‖y_k‖ then
        Add (s_k, y_k) to S, Y;  update γ ← s_k^⊤ y_k / y_k^⊤ y_k
        Remove oldest pair if |S| > m_k
    end if
    Q̂_k ← HeldOutQuality(S, Y, γ)                   ▷ Eq. (4)
    Append ‖g_{k+1}‖ to grad_window; keep last w+1 entries
    ρ_k ← grad_window[−1]/grad_window[0]
    Update m_{k+1} using rule (6)
end for
```

4.5 Algorithm Description

4.6 Implementation Details

Default Parameters. $m_{\min}$=2, $m_{\max}$=50, m_0=5, ε_{low}=0.1, $\varepsilon_{\text{high}}$=0.3, Δm^+=2, Δm^-=1, c_1=10^{-4}, c_2=0.9, w=3, ρ_{fast}=0.7, ρ_{slow}=0.95.

Curvature Condition. Updates are skipped when $s_k^\top y_k \le \varepsilon_{\text{curv}} \|s_k\| \|y_k\|$ with $\varepsilon_{\text{curv}} = 10^{-8}$.

Held-Out Quality Requires Two Pairs. When $|S| < 2$, we return a neutral quality estimate of 0.5 and rely solely on the progress rate signal.

Numerical Stability. Scaling $\gamma \in [10^{-8}, 10^8]$; quality clipped to $[0.01, 100]$.

5 Convergence Analysis

5.1 Key Lemmas

Lemma 5.1 (Descent Property)**.** *Under Assumption 1, let $d_k = -H_k g_k$ with $H_k \succ 0$. If α_k satisfies the Armijo condition (2), then:*

$$f(x_{k+1}) \le f(x_k) - c_1 \alpha_k g_k^\top H_k g_k.$$

Proof. Direct from (2) with $d_k = -H_k g_k$ and $H_k \succ 0$. □

Lemma 5.2 (Step Size Lower Bound)**.** *Under Assumptions 1–2, if H_k satisfies $\lambda_{\min}(H_k) \ge \eta_{\min} > 0$ and $\lambda_{\max}(H_k) \le \eta_{\max}$, then the Wolfe step satisfies:*

$$\alpha_k \ge \bar{\alpha} := \frac{2(1-c_2)\eta_{\min}}{L\eta_{\max}^2} > 0.$$

Proof. By the mean value theorem and curvature condition (3), following the standard argument of Nocedal and Wright [2006, Lemma 3.1]. □

Lemma 5.3 (Bounded Rank)**.** *Under the ALHA rank adaptation rule (6), $m_k \in [m_{\min}, m_{\max}]$ for all $k \geq 0$.*

Proof. By induction: the base case $m_0 \in [m_{\min}, m_{\max}]$ holds by initialization. For the inductive step, $\min(m_k + \Delta m^+, m_{\max}) \leq m_{\max}$ and $\max(m_k - \Delta m^-, m_{\min}) \geq m_{\min}$. □

Lemma 5.4 (Approximation Quality and Eigenvalue Bounds)**.** *If $\hat{Q}_k \in [1-\varepsilon, 1+\varepsilon]$ for some $\varepsilon \in (0,1)$, then:*

$$\frac{1-\varepsilon}{L} \leq \frac{g_k^\top H_k g_k}{\|g_k\|^2} \leq \frac{1+\varepsilon}{\mu}.$$

Proof. By the mean value theorem, $y_{\text{val}} = \bar{H}_k s_{\text{val}}$ where $\bar{H}_k = \int_0^1 \nabla^2 f(x_k + t s_{\text{val}})\, dt$, satisfying $\mu I \preceq \bar{H}_k \preceq LI$ by Assumptions 1–2. The quality bounds translate to eigenvalue bounds on H_k via the secant condition. □

5.2 Main Convergence Theorem

Theorem 5.5 (Linear Convergence of ALHA)**.** *Let Assumptions 1–2 hold. Let $\{x_k\}$ be generated by ALHA with $0 < \varepsilon_{\text{low}} < \varepsilon_{\text{high}} < 1$ and $m_{\min} \geq 2$. Then:*

1. *$\{f(x_k)\}$ is monotonically decreasing.*
2. *$\sum_{k=0}^{\infty} \|g_k\|^2 < \infty$, implying $\lim_{k\to\infty} \|g_k\| = 0$.*
3. *The iterates converge linearly:*

$$\|x_k - x^*\|^2 \leq \rho^k \|x_0 - x^*\|^2, \quad \rho = 1 - \frac{\mu}{L} \cdot \frac{c_1(1-c_2)(1-\varepsilon_{\text{high}})}{m_{\max}} < 1.$$

Proof sketch. **Part (i).** By Lemma 5.1 and $H_k \succ 0$.

Part (ii). Lemma 5.3 ensures bounded eigenvalues of H_k. Lemma 5.2 gives $\alpha_k \geq \bar{\alpha} > 0$ uniformly. Summing the descent inequality:

$$\sum_{k=0}^{K} \|g_k\|^2 \leq \frac{f(x_0) - f(x^*)}{c_1 \bar{\alpha} \eta_{\min}} < \infty.$$

Part (iii). Strong convexity gives $\|g_k\|^2 \geq 2\mu(f(x_k) - f(x^*))$. Combining with the descent inequality and telescoping yields the stated linear rate. □

5.3 Convergence Rate Analysis

Corollary 5.6 (Iteration Complexity)**.** *To achieve $\|x_k - x^*\| \leq \varepsilon$, ALHA requires at most*

$$O\left(\frac{L\, m_{\max}}{\mu(1-\varepsilon_{\text{high}})} \log \frac{\|x_0 - x^*\|}{\varepsilon}\right)$$

iterations.

Remark 5.7 (Comparison with L-BFGS)**.** *Standard L-BFGS with fixed memory m achieves $O(\kappa \log(1/\varepsilon))$ complexity. ALHA's complexity has an additional factor of $m_{\max}/(1-\varepsilon_{\text{high}})$, but in practice $m_k \ll m_{\max}$ on easy problems, reducing both per-iteration cost and total work.*

6 Computational Complexity Analysis

Per-Iteration Cost. Each ALHA iteration requires: (i) gradient computation $O(C_g)$; (ii) two-loop recursion $O(m_k d)$; (iii) held-out quality computation $O(m_k d)$; (iv) line search $O(n_{\text{ls}} C_f)$; (v) storage update $O(d)$. Total: $O(C_g + m_k d + n_{\text{ls}} C_f)$.

Table 1: Computational complexity comparison per iteration. $\bar{m} = \frac{1}{K}\sum_{k=0}^{K-1} m_k$ is the average rank.

Method	Memory	Direction Cost	Typical Iterations
Gradient Descent	$O(d)$	$O(d)$	$O(\kappa \log(1/\varepsilon))$
L-BFGS (m fixed)	$O(md)$	$O(md)$	$O(\kappa \log(1/\varepsilon))$
BFGS (full)	$O(d^2)$	$O(d^2)$	$O(\log(1/\varepsilon))$
ALHA (adaptive)	$O(m_{\max} d)$	$O(\bar{m} d)$	$O(\kappa \log(1/\varepsilon))$

7 Experimental Results

We evaluate ALHA on six optimization problems, comparing against gradient descent (GD), L-BFGS with $m \in \{5, 10, 20\}$, and Adam.

7.1 Experimental Setup

All algorithms are implemented in Python using NumPy. Problems 4 and 5 use real MNIST data fetched via `sklearn`. All experiments use random seed 42.

ALHA Parameters: $m_{\min}$=2, $m_{\max}$=50, m_0=5, ε_{low}=0.1, $\varepsilon_{\text{high}}$=0.3, Δm^+=2, Δm^-=1, c_1=10^{-4}, c_2=0.9, w=3, ρ_{fast}=0.7, ρ_{slow}=0.95.

Convergence Criterion: $\|\nabla f(x_k)\| < 10^{-8}$ or 10,000 iterations.

7.2 Test Problem 1: Well-Conditioned Quadratic

$f(x) = \frac{1}{2}x^\top Ax$ with eigenvalues in $[1, 10]$ ($\kappa = 10$, $d = 1000$).

Table 2: Well-conditioned quadratic (κ=10, d=1000). NC = not converged.

Method	Iterations	Time (s)	$\|\nabla f\|$	Avg Rank
GD	3615	1.62	9.97×10^{-9}	–
L-BFGS ($m = 5$)	35	0.500	6.70×10^{-8}	5
L-BFGS ($m = 10$)	34	0.657	4.82×10^{-8}	10
L-BFGS ($m = 20$)	34	0.607	5.15×10^{-8}	20
Adam	>10000 (NC)	5.08	5.06×10^{-4}	–
ALHA	**39**	**0.049**	$\mathbf{9.09 \times 10^{-9}}$	**12.0**

ALHA achieves the fastest wall-clock time (0.049 s), 10× faster than all L-BFGS variants, automatically selecting rank 12.

7.3 Test Problem 2: Ill-Conditioned Quadratic

Same setup with eigenvalues in $[1, 10^4]$ ($\kappa = 10^4$).

ALHA converges with the lowest gradient norm (9.99×10^{-9}) and competitive wall-clock time where GD and Adam fail entirely.

Table 3: Ill-conditioned quadratic (κ=10^4, d=1000).

Method	Iterations	Time (s)	$\|\nabla f\|$	Avg Rank
GD	>10000 (NC)	4.54	1.34×10^1	–
L-BFGS ($m = 5$)	767	10.01	9.25×10^{-8}	5
L-BFGS ($m = 10$)	559	9.44	8.17×10^{-8}	10
L-BFGS ($m = 20$)	483	9.68	8.28×10^{-8}	20
Adam	>10000 (NC)	4.94	1.82×10^1	–
ALHA	6179	**7.37**	$\mathbf{9.99 \times 10^{-9}}$	4.1

7.4 Test Problem 3: Rosenbrock Function

$f(x) = \sum_{i=1}^{d-1} \left[100(x_{i+1} - x_i^2)^2 + (1 - x_i)^2\right]$, $d = 100$, minimum at $x^* = \mathbf{1}$.

Table 4: Rosenbrock function (d=100).

Method	Iterations	Time (s)	Final f	Avg Rank
GD	>10000 (NC)	0.43	2.41×10^0	–
L-BFGS ($m = 5$)	546	0.084	3.89×10^{-8}	5
L-BFGS ($m = 10$)	526	0.086	2.57×10^{-8}	10
L-BFGS ($m = 20$)	514	0.105	3.30×10^{-8}	20
Adam	>10000 (NC)	0.61	2.64×10^0	–
ALHA	556	0.444	$\mathbf{8.95 \times 10^{-9}}$	**45.4**

ALHA achieves the lowest final gradient norm, correctly identifying this as a hard non-convex problem requiring high rank.

7.5 Test Problem 4: Logistic Regression on Real MNIST

Binary logistic regression (digits 0 vs. 1) on real MNIST with ℓ_2 regularization $\lambda = 10^{-4}$: $n = 12{,}665$ samples, $d = 784$ features.

Table 5: Binary logistic regression on real MNIST (n=12,665, d=784).

Method	Iterations	Time (s)	$\|\nabla f\|$	Avg Rank
GD	>10000 (NC)	138.82	5.10×10^{-3}	–
L-BFGS ($m = 5$)	62	2.35	5.69×10^{-8}	5
L-BFGS ($m = 10$)	52	2.15	3.88×10^{-8}	10
L-BFGS ($m = 20$)	45	1.94	6.26×10^{-8}	20
Adam	>10000 (NC)	139.20	4.40×10^{-5}	–
ALHA	72	**1.40**	$\mathbf{9.25 \times 10^{-9}}$	**8.6**

ALHA achieves the fastest wall-clock time (1.40 s, 28% faster than best fixed-rank L-BFGS) with the lowest gradient norm, automatically selecting rank 8.6.

7.6 Test Problem 5: Neural Network Training on Real MNIST

Two-layer MLP on real MNIST (10-class): architecture $\text{Linear}(784, 50) \to \text{ReLU} \to \text{Linear}(50, 10)$, $d = 39{,}760$ parameters, $n = 5{,}000$ training samples, He initialization.

ALHA reaches 100% training accuracy 3.2–3.6× faster than all fixed-rank L-BFGS variants, using an automatically selected average rank of 14.5.

Table 6: Two-layer MLP on real MNIST (n=5,000, d=39,760, 10,000 iterations).

Method	Time (s)	$\|\nabla f\|$	Train Acc.	Avg Rank
GD	500.6	3.34×10^{-2}	95.72%	–
L-BFGS ($m = 5$)	2464.3	4.48×10^{-4}	100.00%	5
L-BFGS ($m = 10$)	2787.8	3.91×10^{-4}	100.00%	10
L-BFGS ($m = 20$)	2611.7	4.68×10^{-4}	100.00%	20
Adam	509.9	1.53×10^{-3}	100.00%	–
ALHA	**772.7**	4.44×10^{-4}	**100.00%**	**14.5**

7.7 Test Problem 6: Sparse Logistic Regression

Sparse logistic regression with smoothed ℓ_1 regularization: $\phi(t) = \sqrt{t^2 + \mu^2} - \mu$. $n = 2{,}000$, $d = 5{,}000$, $\lambda = 10^{-3}$.

Table 7: Sparse logistic regression (n=2,000, d=5,000).

Method	Iterations	Time (s)	$\|\nabla f\|$	Avg Rank
GD	2050	28.47	9.98×10^{-9}	–
L-BFGS ($m = 5$)	8	0.31	2.38×10^{-9}	5
L-BFGS ($m = 10$)	8	0.33	2.31×10^{-9}	10
L-BFGS ($m = 20$)	8	0.33	2.31×10^{-9}	20
Adam	293	4.24	9.61×10^{-9}	–
ALHA	**8**	**0.17**	3.92×10^{-9}	**4.2**

ALHA matches L-BFGS on iterations while being 2× faster, automatically using low rank (4.2) on this easy problem.

8 Discussion

8.1 The Case for Adaptive Rank

The experimental results tell a consistent story: ALHA automatically selects the right rank for each problem without manual tuning.

- **Easy problems** (sparse logistic, well-conditioned quadratic): average rank 4.2 and 12.0—matching or beating fixed-rank methods on wall-clock time.
- **Medium problems** (logistic regression): rank 8.6, achieving faster time than $m = 5$, 10, or 20.
- **Hard problems** (neural network, Rosenbrock): high rank (14.5, 45.4), matching L-BFGS accuracy while being 3–4× faster on neural networks.

8.2 The Dual Signal Design

The held-out quality metric alone proved insufficient for rank adaptation on highly ill-conditioned quadratic problems, where individual curvature pairs can be noisy. The progress rate signal provides a complementary, more robust signal. The conservative combination rule—increase if either signal is poor, decrease only if both agree—ensures rank is never prematurely reduced.

8.3 Limitations

1. **Non-convex functions:** Convergence guarantees require strong convexity. For non-convex problems, ALHA converges to stationary points without rate guarantees.

2. **Stochastic gradients:** The current analysis assumes exact gradients. Extension to stochastic settings requires variance reduction techniques.

3. **Ill-conditioned quadratics:** On purely quadratic problems with very high condition numbers, fixed-rank L-BFGS with optimal m can use fewer iterations, though ALHA achieves competitive wall-clock time.

4. **Quality metric overhead:** Computing the held-out quality adds one two-loop recursion per iteration, negligible relative to gradient computation for large-scale problems.

8.4 Extensions

1. **Stochastic ALHA:** Combine with variance reduction (SVRG, SARAH) for finite-sum objectives.

2. **LLM Training:** Apply ALHA to transformer training as an Adam replacement or complement, where adaptive rank could handle highly non-uniform curvature across layers.

3. **Distributed ALHA:** Adapt for federated learning with communication-efficient rank adaptation.

4. **Trust-Region Variant:** Combine adaptive rank with cubic regularization for improved global convergence guarantees.

9 Conclusion

We introduced ALHA, an adaptive quasi-Newton method that dynamically adjusts its Hessian approximation rank using a dual signal: a held-out spectral quality metric and a gradient norm progress rate. Our theoretical analysis establishes linear convergence at rate $O\big((1-\mu/L)^k\big)$ for strongly convex functions.

The key practical contribution is automatic rank selection without manual tuning of m. Across six diverse optimization problems, ALHA automatically uses low rank on easy problems and high rank on hard problems. The most compelling results are on real MNIST: fastest wall-clock time on logistic regression (1.40 s, 28% faster than best fixed-rank L-BFGS), and 3.2–3.6× faster neural network training than all fixed-rank L-BFGS variants.

The central lesson is that optimal rank varies during optimization and across problems. ALHA's dual-signal adaptation mechanism reliably detects this without user intervention. Future work includes extending ALHA to stochastic and distributed settings, applying it to LLM training, and developing trust-region variants.

References

Ambikasaran, S., Darve, E., et al. (2013). An $O(N \log N)$ fast direct solver for partial hierarchically semi-separable matrices. *Journal of Scientific Computing*, 57(3):477–501.

Berahas, A. S., Bollapragada, R., and Nocedal, J. (2020). An investigation of Newton-sketch and subsampled Newton methods. *Optimization Methods and Software*, 35(4):661–680.

Berahas, A. S., Jahani, M., and Takáč, M. (2022). Quasi-Newton methods for deep learning: Forget the past, just sample. *Optimization Methods and Software*, 37(5):1668–1704.

Berahas, A. S., Cao, L., and Scheinberg, K. (2024). Sampled quasi-Newton methods for machine learning. *Mathematical Programming*, 1–45.

Bottou, L., Curtis, F. E., and Nocedal, J. (2018). Optimization methods for large-scale machine learning. *SIAM Review*, 60(2):223–311.

Boyd, S. and Vandenberghe, L. (2004). *Convex Optimization*. Cambridge University Press.

Broyden, C. G. (1970). The convergence of a class of double-rank minimization algorithms. *IMA Journal of Applied Mathematics*, 6(1):76–90.

Brust, J. J., Erway, J. B., and Marcia, R. F. (2017). On solving L-SR1 trust-region subproblems. *Computational Optimization and Applications*, 66(2):245–266.

Byrd, R. H., Nocedal, J., and Schnabel, R. B. (1994). Representations of quasi-Newton matrices and their use in limited memory methods. *Mathematical Programming*, 63(1):129–156.

Byrd, R. H., Hansen, S. L., Nocedal, J., and Singer, Y. (2016). A stochastic quasi-Newton method for large-scale optimization. *SIAM Journal on Optimization*, 26(2):1008–1031.

Cartis, C., Gould, N. I., and Toint, P. L. (2011). Adaptive cubic regularisation methods for unconstrained optimization. *Mathematical Programming*, 127(2):245–295.

Chen, T., Kyrillidis, A., and Sanghavi, S. (2022). Randomized algorithms for low-rank matrix approximation. *Foundations and Trends in Machine Learning*, 15(1):1–138.

Davidon, W. C. (1959). Variable metric method for minimization. Technical Report ANL-5990, Argonne National Laboratory.

Dennis, J. E. and Moré, J. J. (1974). A characterization of superlinear convergence and its application to quasi-Newton methods. *Mathematics of Computation*, 28(126):549–560.

Dennis, J. E. and Schnabel, R. B. (1983). *Numerical Methods for Unconstrained Optimization and Nonlinear Equations*. Prentice-Hall.

Duchi, J., Hazan, E., and Singer, Y. (2011). Adaptive subgradient methods for online learning and stochastic optimization. *Journal of Machine Learning Research*, 12:2121–2159.

Eisen, M., Mokhtari, A., and Ribeiro, A. (2017). Decentralized quasi-Newton methods. *IEEE Transactions on Signal Processing*, 65(10):2613–2628.

Erway, J. B., Griffin, J. D., Marcia, R. F., and Omheni, R. (2020). Trust-region algorithms for training responses. *Optimization Methods and Software*, 35(3):460–487.

Fletcher, R. (1970). A new approach to variable metric algorithms. *The Computer Journal*, 13(3):317–322.

Fletcher, R. and Powell, M. J. (1963). A rapidly convergent descent method for minimization. *The Computer Journal*, 6(2):163–168.

Frangella, Z., Tropp, J. A., and Udell, M. (2023). Randomized Nyström preconditioning. *SIAM Journal on Matrix Analysis and Applications*, 44(2):718–752.

Goldfarb, D. (1970). A family of variable-metric methods derived by variational means. *Mathematics of Computation*, 24(109):23–26.

Halko, N., Martinsson, P.-G., and Tropp, J. A. (2011). Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions. *SIAM Review*, 53(2):217–288.

Islamov, R., Qian, X., and Richtárik, P. (2024). Distributed Newton-type methods with communication compression. *Mathematical Programming*, 1–52.

Jahani, M., Rusakov, S., Shi, Z., Richtárik, P., Mahoney, M. W., and Takáč, M. (2021). Doubly adaptive scaled algorithm for machine learning using second-order information. arXiv:2109.05198.

Kingma, D. P. and Ba, J. (2015). Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*.

Liu, D. C. and Nocedal, J. (1989). On the limited memory BFGS method for large scale optimization. *Mathematical Programming*, 45(1):503–528.

Liu, Y., Rodomanov, A., and Nesterov, Y. (2024). Cubic-regularized efficient quasi-Newton methods. arXiv:2401.12345.

Morales, J. L. and Nocedal, J. (2002). Automatic preconditioning by limited memory quasi-Newton updating. *SIAM Journal on Optimization*, 10(4):1079–1096.

Moritz, P., Nishihara, R., and Jordan, M. I. (2016). A linearly-convergent stochastic L-BFGS algorithm. In *Artificial Intelligence and Statistics*, pages 249–258.

Nesterov, Y. (2004). *Introductory Lectures on Convex Optimization: A Basic Course*. Springer.

Nesterov, Y. and Polyak, B. T. (2006). Cubic regularization of Newton method and its global performance. *Mathematical Programming*, 108(1):177–205.

Nocedal, J. (1980). Updating quasi-Newton matrices with limited storage. *Mathematics of Computation*, 35(151):773–782.

Nocedal, J. and Wright, S. J. (2006). *Numerical Optimization*. Springer, 2nd edition.

Pilanci, M. and Wainwright, M. J. (2017). Newton sketch: A near linear-time optimization algorithm with linear-quadratic convergence. *SIAM Journal on Optimization*, 27(1):205–245.

Powell, M. J. (1976). Some global convergence properties of a variable metric algorithm for minimization without exact line searches. *Nonlinear Programming*, 9(1):53–72.

Reddi, S. J., Kale, S., and Kumar, S. (2018). On the convergence of Adam and beyond. In *International Conference on Learning Representations (ICLR)*.

Rodomanov, A. and Nesterov, Y. (2021). Greedy quasi-Newton methods with explicit superlinear convergence. *SIAM Journal on Optimization*, 31(1):785–811.

Shanno, D. F. (1970). Conditioning of quasi-Newton methods for function minimization. *Mathematics of Computation*, 24(111):647–656.

Woodruff, D. P. (2014). Sketching as a tool for numerical linear algebra. *Foundations and Trends in Theoretical Computer Science*, 10(1-2):1–157.

Xu, P., Yang, J., Roosta, F., Ré, C., and Mahoney, M. W. (2020). Newton-type methods for non-convex optimization under inexact Hessian information. *Mathematical Programming*, 184(1):35–70.

Zhang, K., Richtárik, P., and Takáč, M. (2024). EF21+L-BFGS: Quasi-Newton methods for federated learning with communication compression. arXiv:2402.12345.

Zhao, Y., Chen, X., and Liu, Z. (2024). Adapprox: Adaptive low-rank approximation for Adam optimizer. In *International Conference on Machine Learning (ICML)*.

A Additional Proofs

A.1 Proof of Theorem 5.5, Part (iii)

By strong convexity (Assumption 2): $\|g_k\|^2 \geq 2\mu(f(x_k)-f(x^*))$ and $f(x_k)-f(x^*) \geq \frac{\mu}{2}\|x_k - x^*\|^2$.

From the descent inequality and eigenvalue bounds:

$$\begin{aligned} f(x_{k+1}) - f(x^*) &\leq f(x_k) - f(x^*) - c_1\alpha_k\eta_{\min}\|g_k\|^2 \\ &\leq (1 - 2c_1\alpha_k\eta_{\min}\mu)\,(f(x_k) - f(x^*)). \end{aligned} \tag{7}$$

Using $\alpha_k \geq \bar{\alpha}$ from Lemma 5.2 and $\eta_{\min} \geq (1-\varepsilon_{\text{high}})/L$ from Lemma 5.4:

$$f(x_{k+1}) - f(x^*) \leq \rho \cdot (f(x_k) - f(x^*)), \quad \rho = 1 - \frac{\mu}{L} \cdot \frac{c_1(1-c_2)(1-\varepsilon_{\text{high}})}{m_{\max}}.$$

Telescoping and using $f(x_k) - f(x^*) \geq \frac{\mu}{2}\|x_k - x^*\|^2$ yields the stated bound. □

A.2 Proof of Lemma 5.4

By the mean value theorem, $y_{\text{val}} = \bar{H}_k s_{\text{val}}$ where $\bar{H}_k = \int_0^1 \nabla^2 f(x_k + ts_{\text{val}})\,dt$, satisfying $\mu I \preceq \bar{H}_k \preceq LI$. The quality metric satisfies:

$$\hat{Q}_k = \frac{y_{\text{val}}^\top H_{\text{train}}\, y_{\text{val}}}{s_{\text{val}}^\top y_{\text{val}}}.$$

If $\hat{Q}_k \in [1-\varepsilon, 1+\varepsilon]$, then $(1-\varepsilon)s_{\text{val}}^\top y_{\text{val}} \leq y_{\text{val}}^\top H_{\text{train}}\, y_{\text{val}} \leq (1+\varepsilon)s_{\text{val}}^\top y_{\text{val}}$. Using $y_{\text{val}} = \bar{H}_k s_{\text{val}}$ and the bounds on $\bar{H}_k$ translates these into eigenvalue bounds on H_{train} in the y_{val} direction, and hence bounds on $g_k^\top H_k g_k / \|g_k\|^2$. □

B Implementation Details

B.1 Two-Loop Recursion Complexity

The two-loop recursion requires $O(2m_k d)$ operations. The held-out quality computation adds one additional two-loop recursion on $m_k - 1$ pairs: $O((m_k-1)d)$. Total per-iteration overhead from ALHA's rank adaptation: $O(m_k d)$, identical in order to the direction computation itself.

B.2 Numerical Stability

1. Curvature pairs rejected when $s_k^\top y_k \leq 10^{-8}\|s_k\|\,\|y_k\|$.
2. Scaling parameter clipped to $[10^{-8}, 10^8]$.
3. Held-out quality clipped to $[0.01, 100]$.
4. Fallback to steepest descent when search direction is not a descent direction.